



CURSO INTEGRADOR II · OCTUBRE 2026

Cómo funciona la plataforma de monitoreo fitosanitario

Arquitectura, tecnologías, flujo, código, base de datos, despliegue y pendientes, explicados para quien nunca usó este stack.

Agrícola Riachuelo S.A.C. · Ica, Perú

monitoreo.agricolariachuelo.org

Qué vamos a ver

Cada bloque empieza desde cero; al final hay un glosario con todos los términos.

#	Bloque	En pocas palabras
1	El problema y la idea	Qué hace el sistema y quién lo usa
2	Conceptos básicos	Frontend, backend, API, dominio, servidor, Docker, Git
3	Arquitectura	Las piezas y cómo se conectan
4	Flujo completo	El viaje de una foto, desde el campo hasta el WhatsApp
5	Tecnologías	Lenguajes, frameworks y servicios, uno por uno
6	App móvil (demo)	Qué hace hoy y qué le falta
7	Código de la plataforma web	Carpetas y archivos principales
8 · 9	API y base de datos	Cómo hablan app y servidor; qué se guarda
10 · 11	Backend y despliegue	Procesos, servidor, GitHub y operación
12	Pendientes y glosario	Lo que falta y el diccionario de términos

1



El problema y la idea

Qué resuelve el sistema, en qué consiste y quién lo usa.

¿Qué problema resolvemos?

Revisar la salud de miles de plantas de vid a simple vista es lento, depende de la memoria y deja poco registro.

Hoy

El personal recorre las hileras y anota lo que ve. Es difícil saber **dónde exactamente** estaba el problema y comparar con semanas anteriores.

La propuesta

Una mochila con celulares toma **fotos de cada hilera**, con su lote, hilera, lateral y posición GPS.

La ayuda de la IA

Un modelo de visión (YOLO) **sugiere** en qué fotos hay indicios de plaga o enfermedad.

La decisión humana

Un **especialista** revisa la foto en la web y decide. Si confirma, se avisa por WhatsApp al jefe de fundo.

 **Dato clave:** la IA nunca decide sola: solo ordena el trabajo del especialista para que revise primero lo más sospechoso.

El sistema en una frase

«Del campo a la decisión»: fotos ordenadas por ubicación, sugeridas por IA y validadas por una persona.

App móvil

Captura las fotos en el campo, incluso sin internet, y luego las envía.

Plataforma web

Recibe las fotos, las guarda, las analiza y permite revisarlas en un navegador.

Servicios en la nube

Base de datos (Supabase), fotos (Cloudinary) y avisos (WhatsApp).

 **Para entenderlo:** piensa en un hospital: la app es la ambulancia que trae los casos, la web es la sala de diagnóstico y la nube es el archivo de historias clínicas.

¿Quién usa qué?

Cada persona tiene un rol; el rol decide qué pantallas y acciones ve.

Rol	Dónde trabaja	Qué hace
Operador de campo	App móvil	Lleva la mochila, inicia la sesión de monitoreo y recorre las hileras
Especialista fitosanitario	Web	Revisa las fotos sugeridas y decide: confirmar, descartar o pedir más evidencia
Supervisor	Web	Ve el panel, el mapa y los reportes; sigue el avance
Administrador	Web	Aprueba cuentas, gestiona catálogos, modelos de IA y destinatarios
Jefe de fundo / supervisor de zona	WhatsApp	Recibe el aviso solo cuando el especialista confirma un caso

2



Conceptos básicos

Las 6 ideas que hay que tener claras antes de ver la arquitectura. Sin código.

Frontend, backend y base de datos

Casi todo sistema se divide en tres capas que cumplen funciones distintas.

Frontend (lo que se ve)

Pantallas, botones y formularios. En nuestro caso: la **web** en el navegador y la **app** en el celular.

Backend (el que piensa)

Programa que corre en un servidor: valida, aplica reglas, decide qué guardar. Aquí es **Django**.

Base de datos (la memoria)

Guarda todo de forma ordenada en tablas: usuarios, sesiones, fotos, decisiones. Aquí es **PostgreSQL en Supabase**.

 **Para entenderlo:** un restaurante. El frontend es el salón y el mesero, el backend es la cocina que prepara según las recetas, y la base de datos es la despensa.

¿Qué es una API?

Una API es un «menú» de pedidos que un programa acepta. La app no entra a la base de datos: le pide cosas al backend por la API.

📍 La app pide

```
GET https://monitoreo.agricolariachuelo.org
/api/v1/health
```

Cada pedido tiene una **dirección** (URL) y un **verbo**: GET para leer, POST para enviar datos.

✅ El backend responde

```
{
  "status": "UP",
  "serverTime": "2026-10-05T22:02:24Z"
}
```

La respuesta llega en **JSON**: texto con pares «nombre: valor» que cualquier lenguaje entiende.

📌 **Dato clave:** nuestra API se llama `/api/v1`. El «v1» es la versión: si algún día cambia de forma incompatible, se crea `/api/v2` sin romper la app que ya está instalada.

Dominio, DNS y HTTPS

Así encuentra el navegador nuestro servidor y por qué la conexión es segura.

Dominio

El nombre fácil de recordar:

`monitoreo.agricolariachuelo.org` . Lo compramos en Hostinger.

DNS

La «guía telefónica» de internet: traduce el nombre a la dirección numérica del servidor (179.198.103.82).

HTTPS

Conexión cifrada: nadie en el camino puede leer contraseñas ni fotos. El candado del navegador lo confirma.

Certificado

Documento digital que prueba que el sitio es quien dice ser. Lo emite gratis **Let's Encrypt** y se renueva solo.

Servidor, VPS y contenedores

El backend necesita una computadora encendida las 24 horas, con internet y una dirección fija.

VPS

«Servidor privado virtual»: una computadora alquilada en un centro de datos (Hostinger). La administramos por comandos.

Docker y contenedores

Un contenedor es una **caja cerrada** con el programa y todo lo que necesita. Funciona igual en cualquier servidor.

Proxy (Traefik)

El «portero» del servidor: recibe todo lo que llega por internet, pone el HTTPS y lo envía al contenedor correcto.

Aislamiento

Nuestros contenedores tienen límites de memoria y permisos mínimos: no afectan a otros servicios del VPS.

✧ **Para entenderlo:** el VPS es un edificio; cada contenedor es un departamento con su propia llave; Traefik es el portero que recibe las visitas y las lleva al departamento correcto.

Git y GitHub

La forma en que el equipo guarda, comparte y publica el código.



Git

Guarda el **historial** del código: cada cambio (commit) queda registrado con autor, fecha y descripción. Se puede volver atrás.



GitHub

El lugar en internet donde vive el repositorio:
`0skitarBB/agricola-riachuelo-platform`, rama **prueba2**.



push

Subir tus cambios de tu PC a GitHub.



pull

Bajar a tu PC (o al servidor) los cambios que están en GitHub.

⚠ Importante: los secretos (contraseñas, claves de Cloudinary y Supabase) **nunca** se suben a GitHub. El repositorio es público y el archivo `.gitignore` los excluye.

3

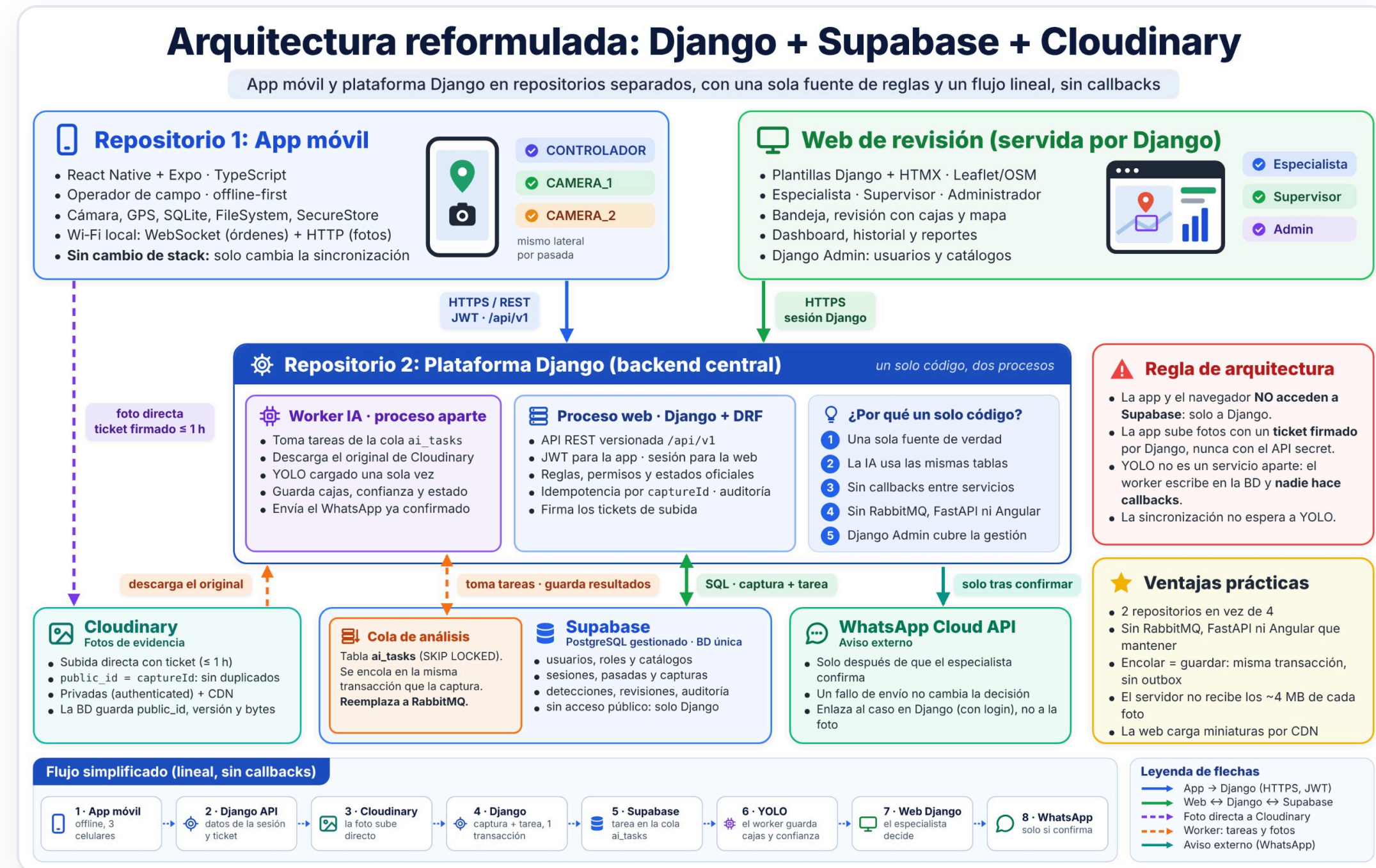


Arquitectura

Las piezas del sistema, cómo se comunican y las reglas que no se rompen.

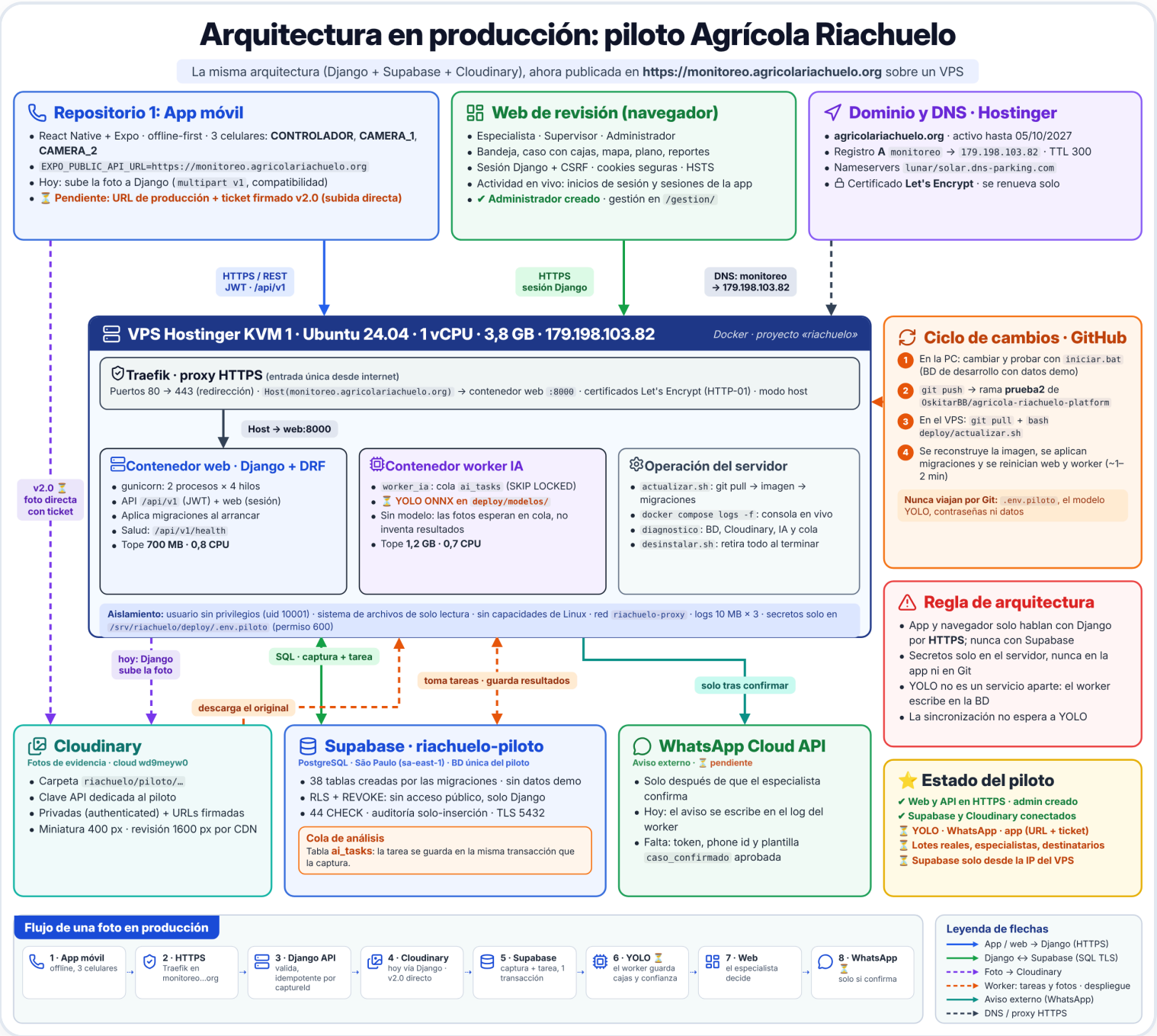
La arquitectura lógica

Qué piezas existen y qué hace cada una. Es el «plano» del sistema.



La arquitectura en producción

El mismo plano, ahora publicado en internet sobre un servidor real.



Dos repositorios, un sistema

El código está separado en dos proyectos que se comunican solo por la API.

Repositorio 1 · App móvil

- `agricola-riachuelo-mobile`
- React Native + Expo + TypeScript
- Lo usa el operador de campo
- Funciona sin internet y sincroniza después



Repositorio 2 · Plataforma

- `agricola-riachuelo-platform`
- Python + Django (web, API y worker)
- Lo usan especialistas, supervisores y admins
- Único que habla con la base de datos

 **Dato clave:** antes se pensaban 4 proyectos (app, API, IA y web por separado). Se redujo a 2: menos cosas que mantener y una sola fuente de reglas.

Las reglas de la arquitectura

Son pocas, pero protegen la seguridad y la consistencia de los datos.



Nadie toca la base de datos excepto Django

Ni la app ni el navegador conocen la contraseña de Supabase. Todo pasa por la API o la web.



Los secretos viven solo en el servidor

Claves de Cloudinary, Supabase y WhatsApp están en un archivo del VPS, no en la app ni en GitHub.



YOLO no es un servicio aparte

El análisis lo hace un proceso del mismo backend (el worker) que escribe directo en la base.



Nadie espera a la IA

La app termina de sincronizar aunque la IA todavía no haya analizado la foto.

4



Flujo completo

El recorrido de una foto: desde que se toma en la hilera hasta que llega el aviso.

El viaje de una foto (1/2)

Del campo al servidor.

1



Se toma la foto

El controlador ordena a las 2 cámaras disparar. Cada foto lleva lote, hilera, lateral, marcador y GPS.

2



Se guarda en el celular

Queda en la base local (SQLite) con un identificador único: el `captureId`.

3



Hay internet: se envía

La app manda los datos de la sesión y la foto a la API por HTTPS.

4



La foto va a Cloudinary

Hoy Django la recibe y la sube; en la versión 2.0 la app la sube directo con un permiso temporal.

El viaje de una foto (2/2)

Del servidor a la decisión.

5



Se registra y se encola

Supabase guarda la captura y, en la **misma operación**, una tarea en la cola de análisis.

6



La IA la analiza

El worker toma la tarea, descarga la foto y YOLO marca cajas con su confianza. Si hay indicio, abre un caso.

7



El especialista decide

En la bandeja web ve la foto con las cajas y elige: confirmar, descartar o evidencia insuficiente.

8



Aviso por WhatsApp

Solo si confirma: el worker envía el mensaje al jefe de fundo con el enlace al caso.

¿Y si no hay internet en el campo?

La app está pensada como **offline-first**: primero guarda, después envía.

Sin señal

La app sigue funcionando: guarda sesiones y fotos en el celular. Muestra cuántas faltan enviar.

Vuelve la señal

Envía todo lo pendiente en orden. Si se corta a la mitad, lo vuelve a intentar.

Sin duplicados

Cada foto tiene su `captureId`. Si llega dos veces, el servidor la reconoce y no la repite (idempotencia).

Login sin internet

Tras el primer ingreso con internet, el operador puede entrar sin señal hasta 7 días.

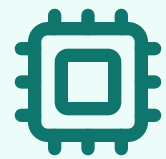
Qué pasa cuando el especialista decide

Cada decisión cambia el estado del caso y queda registrada para siempre.

Decisión	Estado del caso	¿Avisa por WhatsApp?
Confirmar	CONFIRMADO_POR_ESPECIALISTA	Sí: al jefe de fundo y supervisor de zona del lote
Descartar	DESCARTADO	No
Evidencia insuficiente	EVIDENCIA_INSUFICIENTE	No (se puede volver a revisar en campo)
(antes de decidir)	PENDIENTE_REVISION	No

 **Dato clave:** toda decisión se guarda en la **auditoría**: quién decidió, cuándo y qué cambió. Esa tabla no se puede editar ni borrar.

5



Tecnologías

Cada herramienta del stack: qué es y para qué la usamos. Una por diapositiva.

El mapa del stack

Una vista rápida antes de ver cada pieza.

Capa	Tecnología	Para qué
App móvil	React Native + Expo · TypeScript · SQLite	Capturar en campo, funcionar offline
Backend	Python 3.12 · Django 5.2 · Django REST Framework	Reglas, API, web y procesos
Web	Plantillas Django · HTMX · Leaflet/OpenStreetMap	Pantallas de revisión, mapa y plano
Base de datos	PostgreSQL en Supabase	Guardar todo de forma segura
Fotos	Cloudinary	Almacenar y mostrar las imágenes
IA	YOLO exportado a ONNX	Detectar indicios en las fotos
Avisos	WhatsApp Cloud API (Meta)	Notificar casos confirmados
Servidor	VPS Ubuntu · Docker · Traefik · gunicorn	Publicar todo en internet con HTTPS

Python y Django

Python es el lenguaje del backend. Django es un framework: un «kit» que ya trae resuelto lo común.

Qué es Django

Un framework web de Python «con baterías incluidas»: usuarios, seguridad, formularios y panel de administración listos.

ORM

Permite trabajar con la base de datos usando Python, sin escribir SQL a mano. Cada clase del código es una tabla.

Django Admin

Un panel automático en `/gestion/` para administrar usuarios, catálogos y modelos de IA.

Seguridad de fábrica

Protección contra ataques comunes (CSRF, inyección SQL), contraseñas con hash y cookies seguras.

Django REST Framework y JWT

Es la parte de Django que atiende a la app móvil.



DRF

Extensión de Django para construir APIs: recibe JSON, valida los datos y responde JSON.



JWT (token)

Al iniciar sesión, la app recibe un «pase» firmado. Lo envía en cada pedido para demostrar quién es.



Dos tokens

Access: dura 15 minutos. **Refresh:** dura 14 días y sirve para pedir un access nuevo sin volver a escribir la contraseña.



Documentación automática

La API se describe sola en `/api/v1/schema` (formato OpenAPI), útil para el equipo de la app.



Para entenderlo: el JWT es como la pulsera de un evento: te la dan en la entrada (login), la muestras en cada puerta y caduca a cierta hora.

La web: plantillas, HTMX y Leaflet

La web se arma en el servidor y se actualiza por partes, sin un framework pesado de JavaScript.

Plantillas Django

Archivos HTML con «huecos» que Django rellena con datos antes de enviarlos al navegador.

HTMX

Pequeña librería que actualiza **solo un pedazo** de la página (por ejemplo, la bandeja cada 15 s) sin recargar todo.

Leaflet + OpenStreetMap

Dibuja el mapa del fondo con los casos de colores según su estado.

Detalles de uso

Sonidos, atajos de teclado, avisos emergentes y pantallas de carga, hechos con JavaScript propio (sin CDN).

PostgreSQL y Supabase

PostgreSQL es la base de datos; Supabase es la empresa que nos la da administrada en la nube.

PostgreSQL

Base de datos relacional: guarda la información en tablas que se relacionan entre sí (un lote tiene hileras, etc.).

Supabase

La aloja, hace copias y la mantiene. Usamos **solo la base de datos**; no usamos su API pública, Auth ni Storage.

Región São Paulo

El proyecto del piloto está en Sudamérica para responder rápido al servidor.

Conexión segura

Django se conecta por el «Session pooler», puerto 5432, con cifrado TLS.

Cloudinary

Servicio especializado en guardar y entregar imágenes.

Por qué no guardar fotos en la base

Las fotos pesan ~4 MB cada una. La base solo guarda su identificador; el archivo vive en Cloudinary.

Fotos privadas

Se suben como «authenticated»: solo se ven con un enlace firmado que genera Django.

Tamaños automáticos

Cloudinary genera la **miniatura** (400 px) y la versión de **revisión** (1600 px) y las sirve rápido por CDN.

Orden

Cada foto se guarda en `riachuelo/piloto/sesión/pasada/captura`; el nombre es su captureid, así no se duplica.

YOLO y ONNX: la IA

YOLO es un modelo de visión por computadora que encuentra objetos en una imagen.

Qué devuelve

Para cada foto: **cajas** alrededor de lo sospechoso, la **clase** (qué cree que es) y su **confianza** (0 a 100 %).

ONNX

Formato estándar para ejecutar el modelo entrenado en el servidor, sin GPU y sin instalar todo el entorno de entrenamiento.

Umbrales

Solo se abren casos si la confianza supera un umbral configurable desde la web.

Estado actual

El servidor ya está listo para cargarlo. **Falta el modelo entrenado.**

 **Pendiente:** sin modelo, las fotos se quedan «en cola». No se inventan resultados; cuando se copie el modelo, se analizan solas.

WhatsApp Cloud API

La forma oficial de Meta para que un sistema envíe mensajes de WhatsApp.

Cuándo se envía

Solo cuando el especialista **confirma** un caso. Nunca por una sugerencia de la IA.

A quién

A los destinatarios del lote (jefe de fundo, supervisor de zona) que dieron su consentimiento.

Plantilla aprobada

Meta exige una plantilla aprobada (`caso_confirmado`) con lote, hilera, lateral, ubicación, decisión y enlace.

Estado actual

Hoy el mensaje se escribe en el registro del worker. Falta el token, el número y la plantilla.

Docker, Traefik y gunicorn

Las herramientas que mantienen el backend funcionando en el servidor.

Docker

Empaqueta la plataforma en una «imagen». De ella salen 2 contenedores: **web** y **worker**.

gunicorn

El programa que atiende las visitas a Django dentro del contenedor web: 2 procesos con 4 hilos cada uno.

Traefik

El portero del VPS: recibe el tráfico en los puertos 80/443, pone el HTTPS y lo pasa al contenedor web.

Límites

Web: máx. 700 MB y 0,8 CPU. Worker: máx. 1,2 GB y 0,7 CPU. Nunca acaparan el servidor.

React Native y Expo: la app

Tecnología para hacer apps de celular con un solo código para Android e iPhone.

React Native

Se programa en **TypeScript** (JavaScript con tipos) y se convierte en una app nativa.

Expo

Herramientas que simplifican cámara, GPS, almacenamiento seguro y la generación del instalador (APK).

SQLite

Una base de datos pequeña dentro del celular, para trabajar sin internet.

Red local

Los 3 celulares se hablan por el Wi-Fi de la mochila: WebSocket para órdenes y HTTP para las fotos.

6



App móvil (demo)

Qué hace hoy la app, cómo está organizada y qué le falta para conectarse al piloto.

Cómo trabaja la mochila

Tres celulares cooperan durante el recorrido de cada hilera.

Controlador

Lo lleva el operador. Inicia la sesión y la pasada, marca los marcadores, guarda el GPS y ordena cuándo disparar.

Cámara 1 y Cámara 2

Apuntan a los dos laterales de la hilera. Toman la foto cuando el controlador lo ordena.

Vinculación por QR

Las cámaras se unen al controlador escaneando un código QR; luego se hablan por el Wi-Fi local.

Calidad técnica

Cada foto se revisa (desenfoco, luz). Si sale mal, se repite en el momento.

Estado de la app

La app está construida por fases. Las fases 0 a 3 funcionan; la sincronización con el servidor está pendiente.

Fase	Contenido	Estado
0	Base del proyecto, configuración, estructura por capas, pruebas	Listo
1	Login (con y sin internet), registro, contraseñas, función del celular, permisos	Listo
2	Vinculación por QR, protocolo local (WebSocket + HTTP), simulador de 2 cámaras	Listo
3	Sesión, pasadas, marcadores, captura manual/automática, calidad, cierre	Listo
4	Sincronización con la plataforma web y la base central	Pendiente

 **Importante:** hoy la app usa un **backend simulado** dentro del propio celular (cuentas @demo.pe). Por eso todavía no aparece nada en la web del piloto.

Cómo está organizada la app

Dos carpetas principales: las pantallas y la lógica.

```
agricola-riachuelo-mobile/  
├─ app/                               pantallas (cada archivo = una ruta)  
│   ├─ (auth)/                       login, registro, recuperar contraseña  
│   ├─ (setup)/                     elegir función y dar permisos  
│   ├─ controller/                  sesión, pasada, marcador, sincronizar  
│   └─ camera/                      vista de cámara y disparo  
├─ src/                              lógica, separada por temas  
│   ├─ api/                         pedidos a la plataforma (y el simulado)  
│   ├─ auth/  storage/  sync/  
│   ├─ local-network/  protocol/  
│   └─ config/                     mochila (Wi-Fi local)  
└─ app.json · eas.json              variables EXPO_PUBLIC_*  
                                   nombre, permisos y builds
```

Lo esencial

- `app/` usa **Expo Router**: la carpeta define la navegación.
- `src/api/` tiene los pedidos reales y `mock/` el simulado.
- `src/config/env.ts` lee la URL del servidor.

Qué le falta a la app para el piloto

Tres pasos, en este orden.

1



Apuntar al servidor

Configurar

EXPO_PUBLIC_API_URL=https://monitoreo.agricolariachuelo.org y
EXPO_PUBLIC_USE MOCK_API=0 .

2



Fase 4: sincronización

Enviar sesiones, pasadas, secuencias, incidencias y fotos a la API /api/v1 , con reintentos.

3



Generar el APK

Expo copia esas variables al compilar: hay que crear un instalador nuevo e instalarlo en los 3 celulares.

 **Pendiente:** después, migrar la subida de fotos al **ticket v2.0** (subida directa a Cloudinary). El servidor ya soporta las dos formas.

7



Código de la plataforma web

Cómo están organizadas las carpetas y cuáles son los archivos que todo integrante debe conocer.

La carpeta del proyecto

Repositorio `agricola-riachuelo-platform`: cada carpeta tiene una responsabilidad.

```
agricola-riachuelo-platform/  
├─ config/           configuración general de Django  
├─ api/              API para la app móvil (/api/v1)  
├─ web/              pantallas web: vistas, HTML, CSS, JS  
├─ campo/ cuentas/ monitoreo/ evidencias/  
├─ ia/ revision/ notificaciones/ auditoria/  
│                   «apps» del negocio (una por tema)  
├─ diagnostico/      revisión de configuración y errores  
├─ simulador/         solo en desarrollo (fotos de prueba)  
├─ deploy/           instalación en el servidor (VPS)  
├─ scripts/ docs/     arranque local y documentación  
├─ manage.py         comando principal de Django  
├─ requirements.txt   librerías de Python  
└─ Dockerfile · iniciar.bat
```

Cómo leerla

- **Entrada:** `api/` (celulares) y `web/` (navegador).
- **Negocio:** 8 apps con las reglas y las tablas.
- **Soporte:** diagnóstico, simulador y despliegue.

Anatomía de una «app» de Django

Las 8 apps del negocio tienen la misma forma. Si entiendes una, entiendes todas.

```
monitoreo/
```

```
├─ models.py
```

```
├─ services.py
```

```
├─ admin.py
```

```
├─ migrations/
```

```
├─ apps.py
```

```
└─ tests/
```

QUÉ se guarda: cada clase = una tabla

CÓMO se hace: las reglas del negocio

cómo se ve en el panel /gestion/

historial de cambios de las tablas

registro de la app en Django

pruebas automáticas

La idea clave

- `models.py` describe los datos.
- `services.py` concentra las reglas: la API y la web lo llaman, así las reglas están **en un solo lugar**.
- Las vistas solo reciben el pedido y llaman al servicio.

✦ **Para entenderlo:** `models.py` es el formulario en blanco, `services.py` es el manual de procedimientos y `migrations/` es el registro de cada vez que se cambió el formulario.

Las apps del negocio (1/2)

Cada una cubre un tema del monitoreo.

App	De qué se encarga	Tablas principales
campo	El fundo: lotes, hileras, segmentos y marcadores en campo	field_lots , field_rows , markers
cuentas	Usuarios, roles, celulares registrados, recuperación de contraseña	users , user_roles , devices
monitoreo	Sesiones de recorrido, pasadas por lateral, secuencias e incidencias	monitoring_sessions , monitoring_passes
evidencias	Las fotos (capturas), su calidad y la conexión con Cloudinary	captures , quality_results

Las apps del negocio (2/2)

Del análisis a la decisión y su registro.

App	De qué se encarga	Tablas principales
ia	Modelos de IA, cola de análisis, detecciones y el worker	model_configs , ai_tasks , detections
revision	Casos para revisar y decisiones del especialista	review_cases , human_reviews
notificaciones	Destinatarios por lote y avisos de WhatsApp	notification_recipients , notifications
auditoria	Registro inmutable de todo lo importante que pasa	audit_events

Las puertas de entrada: api/ y web/

Dos formas de llegar al mismo backend: una para celulares, otra para personas en el navegador.

api/	
├ v1/urls.py	la lista de direcciones de la API
├ v1/views.py	qué hace cada dirección
├ v1/serializers.py	valida y traduce el JSON
└ errors.py	formato único de errores
web/	
├ urls.py · views.py	páginas y acciones de la web
├ queries.py	consultas del panel, mapa y reportes
├ permissions.py	quién puede ver qué (por rol)
├ templates/web/	el HTML de cada pantalla
└ static/web/	CSS, JavaScript, íconos, sonidos

Recuerda

- Ninguna de las dos tiene reglas propias: llaman a los `services.py`.
- La API responde JSON; la web responde HTML.

config/ y los archivos de la raíz

La configuración que afecta a todo el proyecto.

Archivo	Para qué sirve
config/settings.py	La configuración central: base de datos, seguridad, Cloudinary, WhatsApp, IA. Lee las variables del <code>.env</code>
config/urls.py	El «índice» de direcciones: <code>/api/v1/</code> , <code>/gestion/</code> , <code>/diagnostico/</code> y la web
config/restricciones.py	Agrega reglas CHECK a la base (estados válidos, rangos de GPS)
manage.py	La puerta de los comandos: <code>migrate</code> , <code>runserver</code> , <code>createsuperuser</code> , <code>diagnostico</code> ...
.env / .env.example	Variables y secretos de cada entorno. <code>.env.example</code> es la plantilla sin secretos
requirements.txt	Lista de librerías de Python con versión fija

Soporte: diagnóstico, simulador y despliegue

Carpetas que no son del negocio, pero hacen la vida más fácil.

Carpeta / comando	Para qué sirve
diagnostico/ → <code>diagnostico</code>	Revisa todo (BD, Cloudinary, IA, cola, cuentas) y dice qué corregir
diagnostico/ → <code>comprobar_bd</code>	Prueba la conexión y explica el error en lenguaje claro
diagnostico/ → <code>respaldo</code> · <code>mantenimiento</code>	Copia de seguridad de la base y limpieza de sesiones vencidas
diagnostico/ (errores)	Los errores del navegador llegan a la consola del servidor con un <code>traceId</code>
simulador/	Solo en desarrollo: imita Cloudinary y genera fotos de prueba
web/ → <code>sembrar_demo</code>	Carga datos de demostración (prohibido en el piloto)
deploy/	Instalar, actualizar y retirar la plataforma del VPS

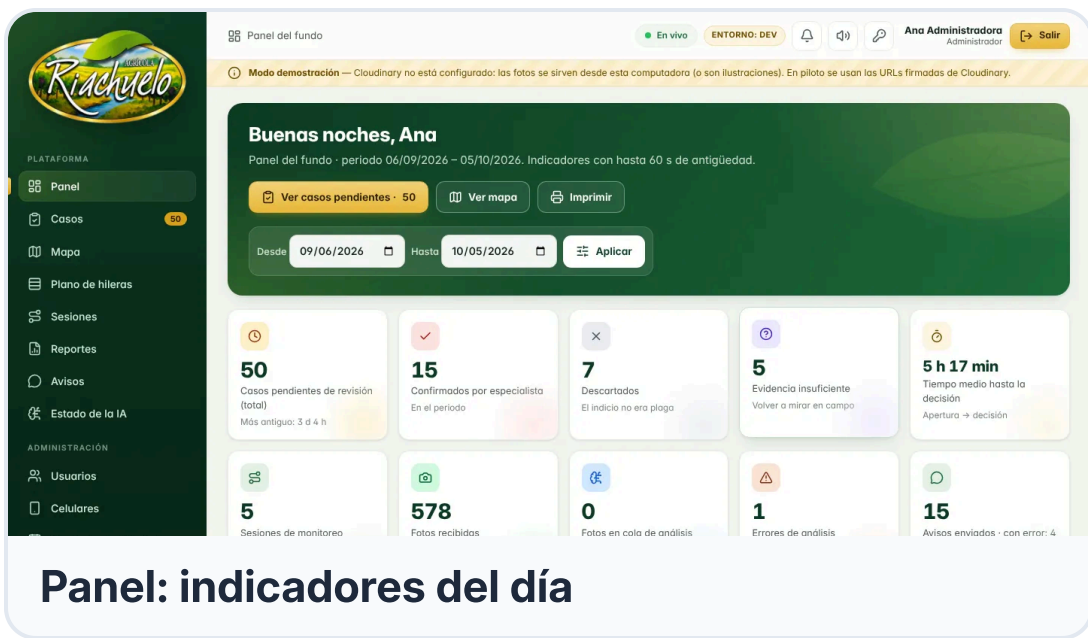
Los 10 archivos que todo integrante debe conocer

Si solo tienes una hora para entender el proyecto, empieza por estos.

#	Archivo	Por qué importa
1	README.md	Cómo instalar y ejecutar
2	config/settings.py	Toda la configuración
3	*/models.py	Qué datos existen (las tablas)
4	*/services.py	Las reglas del negocio
5	api/v1/urls.py	Qué ofrece la API a la app
6	web/urls.py	Qué pantallas tiene la web
7	ia/management/commands/worker_ia.py	Cómo se analizan las fotos
8	docs/ARQUITECTURA_DATOS.md	Diseño y seguridad de la base
9	docs/INTEGRACION_APP.md	Contrato con la app móvil
10	deploy/LEEME.md	Cómo vive en el servidor

Las pantallas de la web

Lo que ven especialistas, supervisores y administradores.



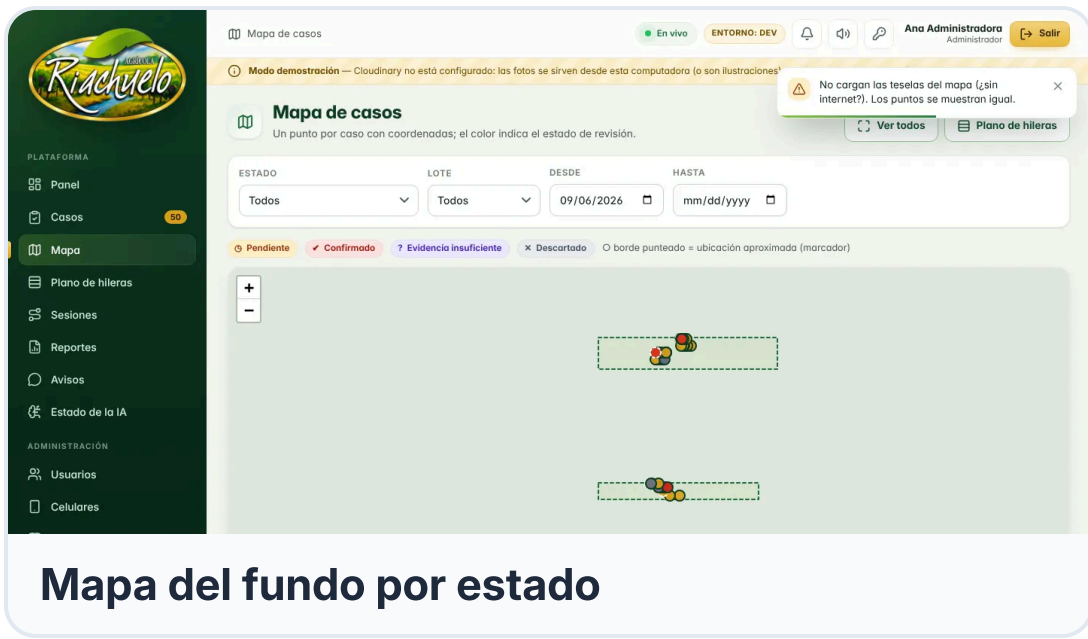
Panel: indicadores del día



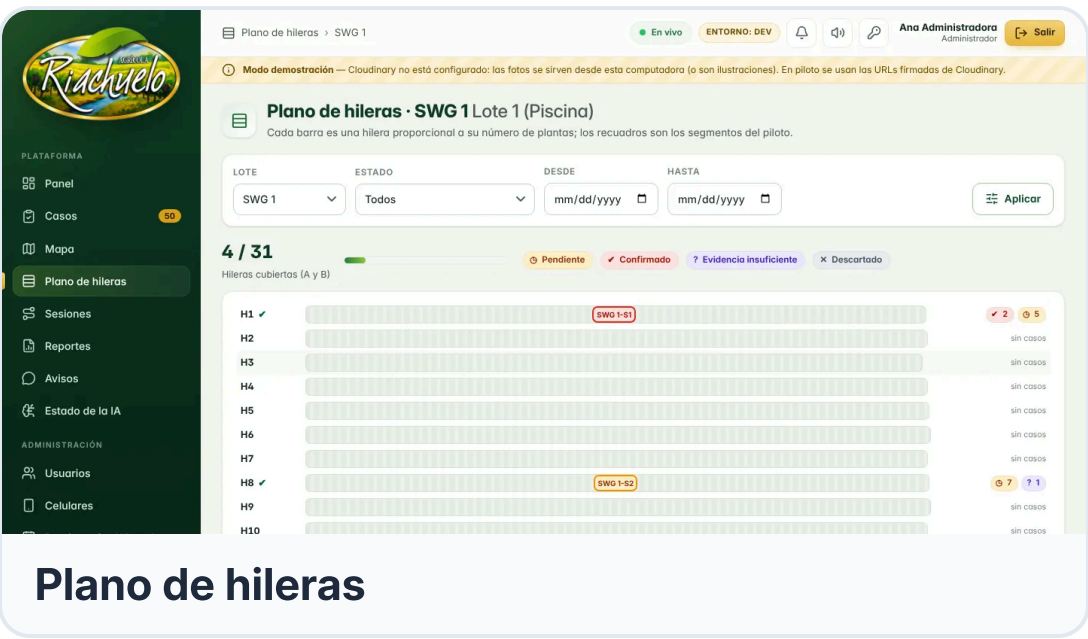
Bandeja: casos por revisar



Caso: foto con cajas y decisión



Mapa del fondo por estado



Plano de hileras



También se adapta al celular

El frontend de la web, archivo por archivo

Todo se sirve desde el propio servidor: no se usan CDN externos.

Archivo	Qué hace
templates/web/base.html	El «marco» común: menú, cabecera, avisos. Las demás pantallas se insertan dentro
templates/web/*.html	Una plantilla por pantalla: bandeja, caso, mapa, plano, reportes, usuarios...
templates/web/partials/	Pedazos que HTMX actualiza solos (tabla de la bandeja, actividad en vivo)
static/web/app.js	Comportamiento general: avisos, sonidos, barra de carga, errores a la consola
static/web/caso.js · mapa.js	Zoom de la foto, cajas y atajos de decisión · mapa con Leaflet
static/web/app.css · vendor/	Estilos propios · copias locales de HTMX y Leaflet

8



La API

Cómo conversan la app móvil y el servidor: direcciones, formato y subida de fotos.

Las direcciones principales de /api/v1

Todas empiezan con `https://monitoreo.agricolariachuelo.org/api/v1`.

Grupo	Direcciones	Para qué
Salud	<code>GET /health</code>	¿El servidor y la base responden?
Cuenta	<code>/auth/register</code> · <code>login</code> · <code>refresh</code> · <code>logout</code> · <code>me</code>	Registro, ingreso y tokens
Arranque	<code>GET /mobile/bootstrap</code>	Catálogos para trabajar offline: lotes, hileras, marcadores
Recorrido	<code>/sessions</code> · <code>/sessions/{id}/passes</code>	Crear o actualizar sesiones y pasadas
Detalle	<code>/sequences/batch</code> · <code>/incidents/batch</code>	Secuencias de disparo e incidencias, en lote
Fotos	<code>/captures/{id}/upload-ticket</code> · <code>/captures/upload</code>	Permiso de subida y confirmación de la foto
Documentación	<code>GET /schema</code>	Descripción automática de toda la API (OpenAPI)

Cómo se ven los mensajes

Siempre JSON, con nombres en camelCase. Los errores tienen siempre la misma forma.

✓ Respuesta correcta (login)

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": { "fullName": "Ana Pérez",
            "roles": ["OPERADOR_CAMPO"] }
}
```

✗ Error (siempre igual)

```
{
  "code": "VALIDATION_ERROR",
  "message": "Revisa los datos.",
  "fieldErrors": [ { "field": "email",
                     "message": "Correo inválido" } ],
  "traceId": "9f2c41..."
}
```

i Dato clave: el `traceId` es un código único de cada pedido. Si algo falla, con ese código se encuentra el error exacto en la consola del servidor.

Dos formas de subir una foto

El servidor acepta las dos, para no romper la app actual mientras se migra.

Hoy · multipart v1

- La app envía la foto **al servidor** (`POST /captures/upload`)
- Django la revisa (tamaño, md5) y la sube a Cloudinary
- Simple, pero el servidor recibe ~4 MB por foto
- Activo con `API_SUBIDA_MULTIPART=true`



Futuro · ticket v2.0

- La app pide un **ticket** firmado (válido ≤ 1 hora)
- Sube la foto **directo a Cloudinary** con ese ticket
- Confirma a Django que terminó
- El servidor no carga las fotos; escala mejor

 **Pendiente:** cuando los 3 celulares tengan la versión con ticket, se pone `API_SUBIDA_MULTIPART=false` en el servidor.

9



Base de datos

Qué se guarda, cómo se relaciona y cómo se protege.

38 tablas en total

Se crean solas con las migraciones; nadie las diseña a mano en Supabase.

26 tablas del negocio

Las de las 8 apps: lotes, usuarios, sesiones, capturas, detecciones, casos, avisos, auditoría.

12 tablas técnicas

Las que Django necesita para funcionar: migraciones aplicadas, sesiones web, permisos, tokens revocados y caché.

Poco espacio

100 000 fotos ocupan ~103 MB en la base, porque las imágenes están en Cloudinary. El plan gratis tiene 500 MB.

Sin acceso público

Solo Django entra, con su usuario y contraseña, por conexión cifrada.

Cómo se relacionan los datos

La cadena principal: del lugar físico hasta el aviso.



✦ **Para entenderlo:** es como un árbol genealógico: cada aviso «desciende» de una revisión, que viene de un caso, que viene de una foto tomada en una pasada de una sesión en una hilera de un lote.

¿Qué es una migración?

Un archivo que describe un cambio en la estructura de la base, en orden.

Se generan desde el código

Al cambiar un `models.py`, Django crea un archivo nuevo en `migrations/` con los cambios.

Se aplican solas

En tu PC con `iniciar.bat`; en el servidor, al arrancar el contenedor web (`migrate`).

Cambios que agregan

Nuevas tablas o columnas: no tocan los datos existentes. Son seguras.

Cambios que quitan

Borrar o renombrar columnas puede perder datos: probar antes y hacer respaldo.

Cómo se protegen los datos

Varias capas, por si una falla.

RLS + REVOKE

Se bloquea el acceso de los usuarios públicos de Supabase. La base no se puede leer desde internet aunque alguien tenga la URL del proyecto.

44 reglas CHECK

La base rechaza estados inválidos o coordenadas imposibles, aunque el código tuviera un error.

Auditoría inmutable

Un «trigger» impide editar o borrar el historial de auditoría.

Contraseñas con hash

Nunca se guarda la contraseña; solo una huella (PBKDF2) que no se puede revertir.

Dos bases de datos separadas

Una para probar y otra para trabajar de verdad. Nunca se mezclan.

Desarrollo

- Proyecto actual en EE. UU.
- Tiene datos de demostración (cuentas `@demo.pe`, 576 fotos de prueba)
- La usa tu PC con `iniciar.bat`
- Se puede romper y volver a cargar



Piloto

- Proyecto `riachuelo-piloto` en São Paulo
- Solo datos reales, sin cuentas demo
- La usa únicamente el servidor (VPS)
- Las migraciones llegan con cada actualización

10



Backend en ejecución

Qué procesos corren, cómo se reparten el trabajo y cómo se configuran.

Un solo código, dos procesos

La misma imagen de Docker arranca de dos maneras distintas.

Proceso web

- `gunicorn`: atiende la web y la API
- Responde en milisegundos
- Nunca analiza fotos ni envía WhatsApp
- Al arrancar aplica las migraciones



Proceso worker

- `python manage.py worker_ia`
- Nunca atiende visitas de internet
- Analiza fotos con YOLO y abre casos
- Envía los avisos de WhatsApp pendientes

 **Dato clave:** así una foto pesada de analizar nunca hace lenta la web: cada proceso tiene su propio trabajo y sus propios límites.

La cola de análisis (sin RabbitMQ)

La cola es simplemente una tabla, `ai_tasks`, dentro de la misma base.

1



Se encola

Cuando llega una foto válida, se crea su tarea en la misma transacción: o se guardan las dos, o ninguna.

2



El worker la toma

Pide la siguiente tarea libre y la «reserva» (SKIP LOCKED) para que nadie más la tome.

3



La procesa

Descarga la foto de Cloudinary, ejecuta YOLO y guarda cajas, confianza y estado.

4



Si falla, reintenta

Un error de red no pierde la tarea: queda para otro intento.



Para entenderlo: es una fila de tickets en una panadería: cada ticket se atiende una sola vez, en orden, y si el panadero se va, el ticket sigue en la fila.

La configuración: el archivo .env

Las mismas líneas de código se comportan distinto según estas variables.

Variable	Qué controla	En el piloto
APP_ENV	Entorno: dev (tu PC) o piloto (servidor)	piloto
DATABASE_URL	Dirección y clave de la base	Supabase São Paulo
CLOUDINARY_URL	Cuenta de Cloudinary	Clave dedicada al piloto
ALLOWED_HOSTS	Dominios aceptados	monitoreo.agricolariachuelo.org
IA_DETECTOR · MODEL_PATH	Qué IA usar y dónde está el modelo	onnx · modelos/modelo.onnx
WHATSAPP_CLIENT	Enviar de verdad o solo escribir en consola	Consola (pendiente)

 **Importante:** en el servidor este archivo se llama `deploy/.env.piloto`, tiene permisos 600 (solo root) y nunca va a GitHub.

Errores y diagnóstico

Cuando algo falla, el sistema lo dice en lenguaje claro.

Una línea por pedido

La consola muestra cada visita con su tiempo y su `traceId`. Avisa si una página es lenta.

Errores del navegador

Si falla un JavaScript o no carga una foto, el navegador lo reporta y aparece en la consola del servidor.

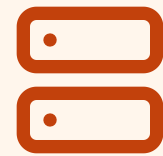
Comando diagnóstico

Revisa base, migraciones, CHECK, Cloudinary, IA, cola y cuentas; dice qué está mal y cómo arreglarlo.

Bloqueos de seguridad

En piloto, Django no arranca si falta Cloudinary, si DEBUG está activo o si el dominio no está configurado.

11



Despliegue y operación

Dónde vive el sistema, cómo se instaló y cómo se actualiza.

Dónde vive cada cosa

Todo lo que hace falta para que funcione en internet.

Pieza	Dónde	Detalle
Dominio	Hostinger	<code>agricolariachuelo.org</code> · registro A <code>monitoreo</code> → 179.198.103.82
Servidor	VPS Hostinger KVM 1	Ubuntu 24.04 · 1 CPU · 3,8 GB de RAM · carpeta <code>/srv/riachuelo</code>
HTTPS	Traefik + Let's Encrypt	Certificado válido, se renueva solo
Base de datos	Supabase	Proyecto <code>riachuelo-piloto</code> , São Paulo
Fotos	Cloudinary	Cloud <code>wd9meyw0</code> , carpeta <code>riachuelo/piloto</code>
Código	GitHub	Rama <code>prueba2</code> del repositorio público

Cómo se instaló (una sola vez)

El script `deploy/instalar.sh` automatiza casi todo.

1



Descargar el código

`git clone` del repositorio en `/srv/riachuelo`.

2



Detectar Traefik

El script encuentra el proxy del VPS y su configuración de HTTPS.

3



Poner los secretos

Se crea `.env.piloto` y se pegan Supabase y Cloudinary con `nano`.

4



Construir y levantar

Se crea la imagen, arrancan web y worker, se aplican migraciones.

5



Primer administrador

`createsuperuser` crea la cuenta para entrar a la web.

El ciclo de un cambio

Desde que alguien edita el código hasta que está en internet.

1



Cambiar y probar

En tu PC, con `iniciar.bat` y la base de desarrollo.

2



Subir

```
git add -A → git commit  
-m "... " → git push.
```

3



Bajar en el servidor

```
cd /srv/riachuelo && git  
pull
```

4



Aplicar

```
bash  
deploy/actualizar.sh:  
reconstruye, migra y reinicia  
(~1-2 min).
```



Importante: antes de empezar a trabajar haz `git pull` en tu PC: varios integrantes suben a la misma rama.

Qué le pasa a cada parte al actualizar

Una actualización normal no pierde datos.

Parte	Qué sucede
Web y API	Se reinicia; queda sin servicio unos 10–30 segundos
Worker	Termina la foto en curso (hasta 30 s) y se reinicia; las tareas siguen en la base
Base de datos	Recibe las migraciones nuevas; los datos existentes se conservan
App móvil	No se entera: si justo sincronizaba, reintenta sin duplicar
Secretos y modelo	No cambian: no viajan por Git

 **Importante:** no se debe romper `/api/v1`: los celulares en campo tienen instalada una versión fija de la app. Agregar campos sí; quitar o renombrar, no.

Comandos del día a día en el servidor

Se escriben en la consola del VPS, dentro de `/srv/riachuelo/deploy`.

```
# ver si todo está en marcha
docker compose ps

# ver la consola en vivo (Ctrl+C para salir)
docker compose logs -f --tail 50 web worker

# revisar la configuración completa
docker compose exec web python manage.py diagnostico

# reiniciar solo el worker
docker compose restart worker

# aplicar cambios del código o del .env.piloto
bash actualizar.sh
```

Para recordar

- **logs** es lo primero que se mira si algo falla.
- **diagnostico** dice qué falta y cómo arreglarlo.
- **desinstalar.sh** retira todo al terminar el proyecto.

Ejecutar el proyecto en tu PC

Para desarrollar o mostrar el sistema sin tocar el piloto.

1



Clonar el repositorio

`git clone` y entrar a la carpeta.

2



Doble clic en iniciar.bat

Crea el entorno de Python, instala librerías y crea el `.env`.

3



Base de datos

Sin `DATABASE_URL` usa SQLite; con la de desarrollo usa Supabase EE. UU.

4



Abrir la web

`http://127.0.0.1:8000` con cuentas demo (`@demo.pe` / `Demo2026`).

ⓘ Dato clave: en tu PC todo está simulado (Cloudinary, IA y WhatsApp) para poder probar el flujo completo sin cuentas externas.

12



Pendientes

Lo que falta para que el piloto funcione de punta a punta.

Pendiente 1: el modelo YOLO

El servidor ya está preparado; falta el modelo entrenado.

1



Exportar

```
yolo export model=best.pt  
format=onnx imgsz=640
```

2



Subir al VPS

Copiar a
`/srv/riachuelo/deploy/mod
elos/modelo.onnx`

3



Registrarlo

En `/gestion/` → Modelos
de IA: clases en orden,
imgsz, umbral de confianza
e IoU

4



Reiniciar

```
bash  
deploy/actualizar.sh : las  
fotos en cola se analizan  
solas
```



Pendiente: se necesita el archivo `.pt` u `.onnx`, la lista de clases, el tamaño de imagen del entrenamiento y los umbrales.

Pendiente 2: la app móvil

Es el paso que conecta el campo con la plataforma.

Fase 4 • sincronización

Enviar sesiones, pasadas, secuencias, incidencias y fotos a `/api/v1`.

URL de producción

`EXP0_PUBLIC_API_URL` + `EXP0_PUBLIC_USE MOCK_API=0` y un APK nuevo.

Cuentas reales

Los operadores se registran en la app y el administrador los aprueba en la web.

Ticket v2.0 (después)

Subida directa de fotos a Cloudinary y luego `API_SUBIDA_MULTIPART=false`.

Pendiente 3: operación y seguridad

Tareas de configuración que no requieren programar.

Tarea	Dónde	Por qué
Cargar lotes, hileras y marcadores reales	Web / <code>/gestion/</code>	Sin lotes la app no puede iniciar sesiones
Crear especialistas y supervisores	Web → Usuarios	Alguien tiene que revisar los casos
Destinatarios de WhatsApp con consentimiento	Web → Destinatarios	A quién avisar por lote
WhatsApp Cloud API: token, número y plantilla	Meta + <code>.env.piloto</code>	Hoy los avisos solo se escriben en consola
Restringir Supabase a la IP del VPS	Panel de Supabase	Nadie más podrá ni intentar conectarse
Respaldo automático de la base	VPS	El plan gratis de Supabase no da copias descargables

Resumen del estado

Dónde estamos hoy (octubre de 2026).

Parte	Estado
Plataforma web y API en HTTPS	Listo en monitoreo.agricolariachuelo.org
Base de datos del piloto con seguridad	Listo 38 tablas, RLS, CHECK, auditoría
Fotos en Cloudinary	Listo clave dedicada y fotos privadas
Despliegue y actualizaciones	Listo GitHub → VPS con un comando
App móvil	Demo fases 0–3; falta sincronizar (fase 4)
IA (YOLO)	Pendiente falta el modelo entrenado
WhatsApp	Pendiente falta la cuenta de Meta
Datos reales (lotes, usuarios)	Pendiente cargar desde la web

Glosario (1/2)

Los términos que aparecen en el proyecto.

Término	Significado simple
API	Menú de pedidos que un programa acepta de otro
Backend / Frontend	La parte que piensa (servidor) / la parte que se ve (pantallas)
JSON	Formato de texto para intercambiar datos: <code>{"nombre": "valor"}</code>
JWT / token	Pase firmado que demuestra quién eres en cada pedido
ORM	Forma de usar la base de datos con código Python en vez de SQL
Migración	Archivo que cambia la estructura de las tablas, en orden
Offline-first	Funciona sin internet y sincroniza cuando vuelve la señal
Idempotencia	Enviar lo mismo dos veces produce el mismo resultado, sin duplicar

Glosario (2/2)

Infraestructura e inteligencia artificial.

Término	Significado simple
VPS	Computadora alquilada en un centro de datos, encendida 24/7
Contenedor (Docker)	Caja aislada con un programa y todo lo que necesita
Proxy (Traefik)	Portero que recibe el tráfico de internet y lo reparte
DNS	Guía que traduce un nombre de dominio a una dirección IP
HTTPS / certificado	Conexión cifrada / documento que prueba la identidad del sitio
Worker	Proceso que hace trabajo en segundo plano (analizar fotos)
Cola (ai_tasks)	Lista ordenada de tareas pendientes, guardada en la base
YOLO / ONNX	Modelo que detecta objetos en imágenes / formato para ejecutarlo

Cierre

Para llevarse

Si recuerdas solo cuatro ideas, que sean estas.

Dos repositorios, una API

La app captura; la plataforma guarda, analiza y muestra.
Solo hablan por `/api/v1`.

La IA sugiere, la persona decide

Ningún aviso sale sin la confirmación de un especialista.

Un solo guardián de los datos

Solo Django entra a la base; los secretos viven solo en el servidor.

Cambiar es seguro

PC → GitHub → VPS con un comando; las migraciones cuidan la base.